

```

1  # Note 12: Support Vector Machine (SVM) Algorithm
2  # =====
3
4  # Import Libraries
5  import pandas as pd
6  from sklearn.svm import SVR
7  import numpy as np
8  import matplotlib.pyplot as plt
9  from sklearn.model_selection import train_test_split
10 from sklearn.svm import SVC
11
12 # =====
13 # 1. Support Vector Regression (SVR)
14 # =====
15
16 model = SVR(kernel='rbf')
17 model.fit(X_train, y_train)
18 y_pred = model.predict(X_test)
19
20 # Extra Points
21 print(df.head())
22 X = df.iloc[:,0:13] # Columns 0 to 12
23
24 # =====
25 # 2. Support Vector Machine (SVM)
26 # =====
27
28 # =====
29 # Dataset Preparing # No Need to Learn
30 mean1 = 55
31 std_dev1 = 10
32 num_samples = 500
33
34 # Generate random values following a normal distribution
35 column1_numbers = np.random.normal(mean1, std_dev1, num_samples)
36
37 # Limit the values to the range 30 to 120
38 column1_numbers = np.clip(column1_numbers, 30, 120)
39
40 # Round values and convert them to integers
41 column1_numbers = np.round(column1_numbers).astype(int)
42
43 # Define the mean and standard deviation for the Farthest_run feature
44 mean2 = 18
45 std_dev2 = 3
46
47 # Generate random values following a normal distribution
48 column2_numbers = np.random.normal(mean2, std_dev2, num_samples)
49
50 # Limit the values to the range 12 to 26
51 column2_numbers = np.clip(column2_numbers, 12, 26)
52
53 # Round values and convert them to integers
54 column2_numbers = np.round(column2_numbers).astype(int)
55
56 # Generate random binary values (0 or 1) representing qualification status
57 column3_numbers = np.random.randint(2, size=num_samples)
58
59 # Set qualification to 1 when Miles_Per_week is greater than the mean
60 column3_numbers[column1_numbers > mean1] = 1
61
62 # =====
63
64 # Create a dictionary containing all columns
65 data = {
66     'Miles_Per_week': column1_numbers,
67     'Farthest_run': column2_numbers,
68     'Qualified_Boston_Marathon': column3_numbers
69 }
70
71 # Create a DataFrame from the dictionary
72 df = pd.DataFrame(data)

```

```

73
74 # Display the generated dataset
75 print(df)
76
77 # Create a scatter plot to visualize the data
78 plt.figure(figsize=(10, 6))
79
80 # Plot Miles_Per_week against Farthest_run
81 # Color points according to qualification status
82 plt.scatter(
83     df['Miles_Per_week'],
84     df['Farthest_run'],
85     c=df['Qualified_Boston_Marathon'],
86     cmap='coolwarm'
87 )
88
89 # Label the x-axis
90 plt.xlabel('Miles Per Week')
91
92 # Label the y-axis
93 plt.ylabel('Farthest Run')
94
95 # Add a title to the plot
96 plt.title('Scatter Plot')
97
98 # Display the color scale legend
99 plt.colorbar(label='Qualified Boston Marathon')
100
101 # Show the plot
102 plt.show()
103
104 # Select the first two columns as input features
105 X = df.iloc[:, 0:2]
106 print(X)
107
108 # Select the third column as the target variable
109 y = df.iloc[:, 2]
110 print(y)
111
112 # Split the dataset into training and testing sets
113 # 80% for training and 20% for testing
114 X_train, X_test, y_train, y_test = train_test_split(
115     X, y, test_size=0.2, random_state=33
116 )
117
118 # Create an SVM model with default parameters
119 model = SVC()
120
121 # Train the model using the training data
122 model.fit(X_train, y_train)
123
124 # Display the model accuracy on the test dataset
125 print(model.score(X_test, y_test))
126
127 # Regularization Parameter (C)
128 # Smaller C allows more classification errors
129 # and creates a wider margin
130 model_reg0 = SVC(C=0.1)
131
132 # Train the model
133 model_reg0.fit(X_train, y_train)
134
135 # Display the model accuracy
136 print(model_reg0.score(X_test, y_test))
137
138 # Create an SVM model with the default C value
139 model_reg1 = SVC(C=1)
140
141 # Train the model
142 model_reg1.fit(X_train, y_train)
143
144 # Display the model accuracy

```

```
145 print(model_reg1.score(X_test, y_test))
146
147 # Create an SVM model with a large C value
148 # Larger C attempts to classify all training points correctly
149 model_reg2 = SVC(C=1000)
150
151 # Train the model
152 model_reg2.fit(X_train, y_train)
153
154 # Display the model accuracy
155 print(model_reg2.score(X_test, y_test))
156
157 # Gamma Parameter
158 # Smaller gamma creates a smoother decision boundary
159 model_gamma0 = SVC(gamma=0.1)
160
161 # Train the model
162 model_gamma0.fit(X_train, y_train)
163
164 # Display the model accuracy
165 print(model_gamma0.score(X_test, y_test))
166
167 # Gamma Parameter
168 # Medium gamma value
169 model_gamma1 = SVC(gamma=1)
170
171 # Train the model
172 model_gamma1.fit(X_train, y_train)
173
174 # Display the model accuracy
175 print(model_gamma1.score(X_test, y_test))
176
177 # Gamma Parameter
178 # Very large gamma may lead to overfitting
179 model_gamma2 = SVC(gamma=1000)
180
181 # Train the model
182 model_gamma2.fit(X_train, y_train)
183
184 # Display the model accuracy
185 print(model_gamma2.score(X_test, y_test))
186
187 # Kernel Function
188 # Use a linear decision boundary
189 model_linear = SVC(kernel='linear')
190
191 # Train the model
192 model_linear.fit(X_train, y_train)
193
194 # Display the model accuracy
195 print(model_linear.score(X_test, y_test))
196
197 # Kernel Function
198 # Use the Radial Basis Function (RBF) kernel
199 model_linear = SVC(kernel='rbf')
200
201 # Train the model
202 model_linear.fit(X_train, y_train)
203
204 # Display the model accuracy
205 print(model_linear.score(X_test, y_test))
```